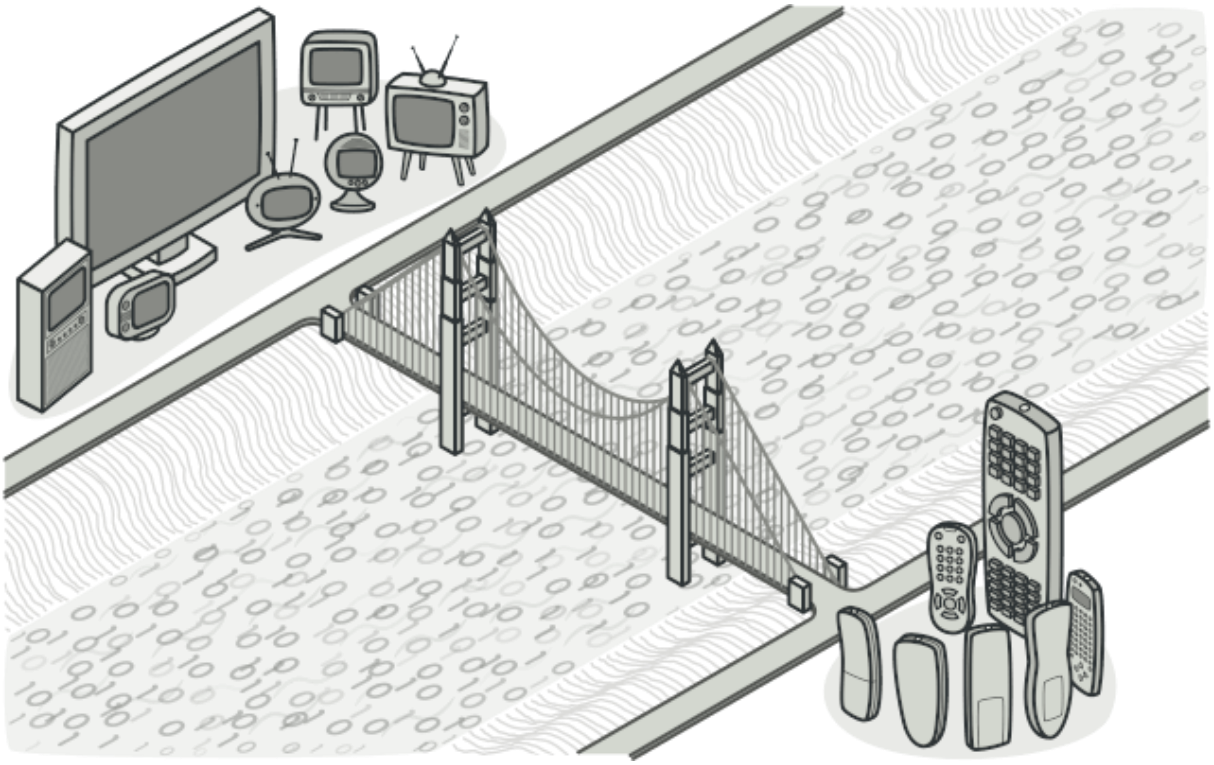
Bridge

Absicht

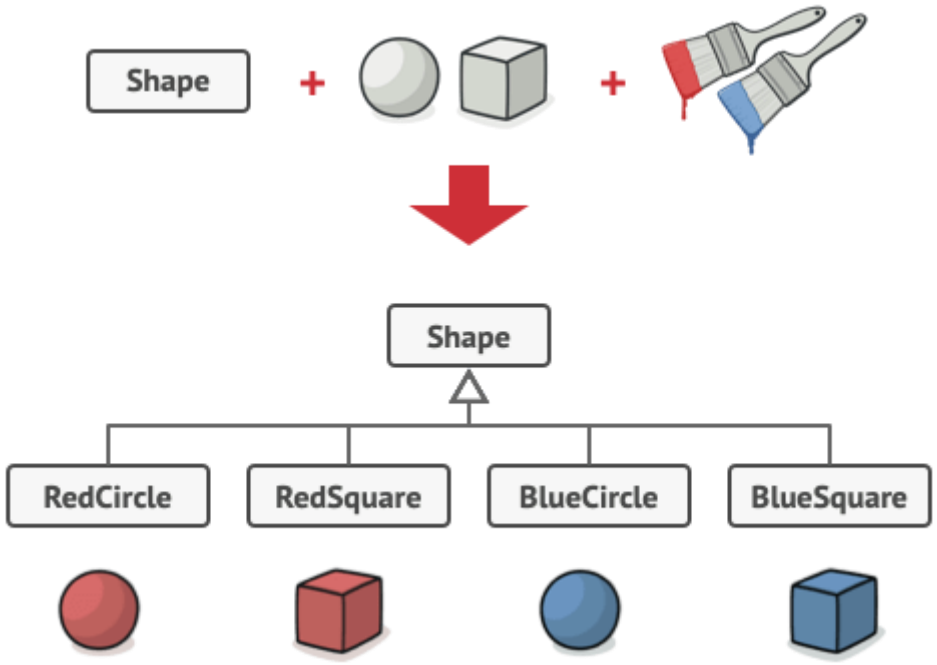
Bridge ist ein strukturelles Entwurfsmuster, mit dem Sie eine große Klasse oder eine Reihe eng verwandter Klassen in zwei separate Hierarchien – Abstraktion und Implementierung – aufteilen können, die unabhängig voneinander entwickelt werden können.



Problem

Abstraktion? Implementierung? Klingt beängstigend? Bleiben Sie ruhig und betrachten wir ein einfaches Beispiel.

Angenommen, Sie haben eine geometrische `Shape`Klasse mit zwei Unterklassen: `Circle`und `Square`. Sie möchten diese Klassenhierarchie um Farben erweitern und planen daher, Unterklassen zu erstellen `Red`und `Blue`zu formen. Da Sie jedoch bereits zwei Unterklassen haben, müssen Sie vier Klassenkombinationen wie `BlueCircle`und erstellen `RedSquare`.



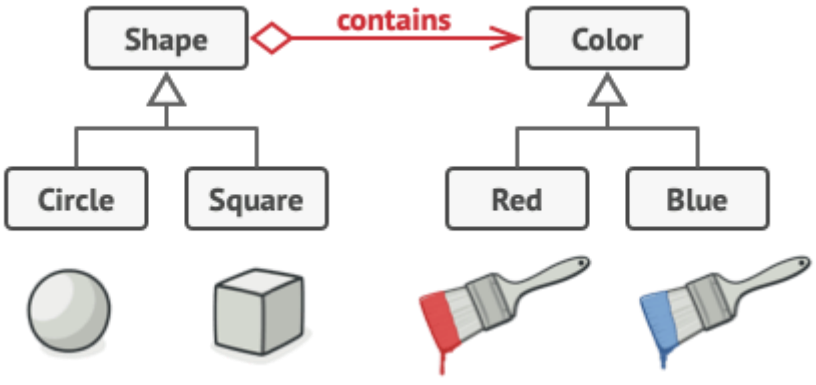
Die Anzahl der Klassenkombinationen wächst geometrisch.

Das Hinzufügen neuer Formtypen und Farben zur Hierarchie lässt diese exponentiell wachsen. Um beispielsweise eine Dreiecksform hinzuzufügen, müssten Sie zwei Unterklassen erstellen, eine für jede Farbe. Und danach müssten für das Hinzufügen einer neuen Farbe drei Unterklassen erstellt werden, eine für jeden Formtyp. Je weiter wir gehen, desto schlimmer wird es.

Lösung

Dieses Problem tritt auf, weil wir versuchen, die Formklassen in zwei unabhängigen Dimensionen zu erweitern: nach Form und nach Farbe. Dies ist ein häufiges Problem bei der Klassenvererbung.

Das Bridge-Muster versucht, dieses Problem zu lösen, indem es von der Vererbung zur Objektkomposition wechselt. Das bedeutet, dass Sie eine der Dimensionen in eine separate Klassenhierarchie extrahieren, sodass die ursprünglichen Klassen auf ein Objekt der neuen Hierarchie verweisen, anstatt alle Zustände und Verhaltensweisen in einer Klasse zu haben.



Sie können die Explosion einer Klassenhierarchie verhindern, indem Sie sie in mehrere verwandte Hierarchien umwandeln.

Mit diesem Ansatz können wir den farbbezogenen Code in eine eigene Klasse mit zwei Unterklassen extrahieren: `Red`und `Blue`. Die `Shape`Klasse erhält dann ein Referenzfeld, das auf eines der Farbobjekte verweist. Nun kann die Form alle farbbezogenen Aufgaben an das verknüpfte Farbobjekt delegieren. Diese Referenz fungiert als Brücke zwischen den Klassen `Shape`und `Color`. Von nun an erfordert das Hinzufügen neuer Farben keine Änderung der Formhierarchie mehr und umgekehrt.

Abstraktion und Implementierung

Das GoF-Buch führt die Begriffe *Abstraktion* und *Implementierung* als Teil der Bridge-Definition ein. Meiner Meinung nach klingen die Begriffe zu akademisch und lassen das Muster komplizierter erscheinen, als es tatsächlich ist. Nachdem wir das einfache Beispiel mit Formen und Farben gelesen haben, wollen wir die Bedeutung hinter den beängstigenden Worten des GoF-Buches entschlüsseln.

Abstraktion (auch *Schnittstelle* genannt) ist eine übergeordnete Steuerungsebene für eine Entität. Diese Ebene sollte keine eigenständige Arbeit leisten. Sie delegiert die Arbeit an die *Implementierungsebene* (auch *Plattform* genannt).

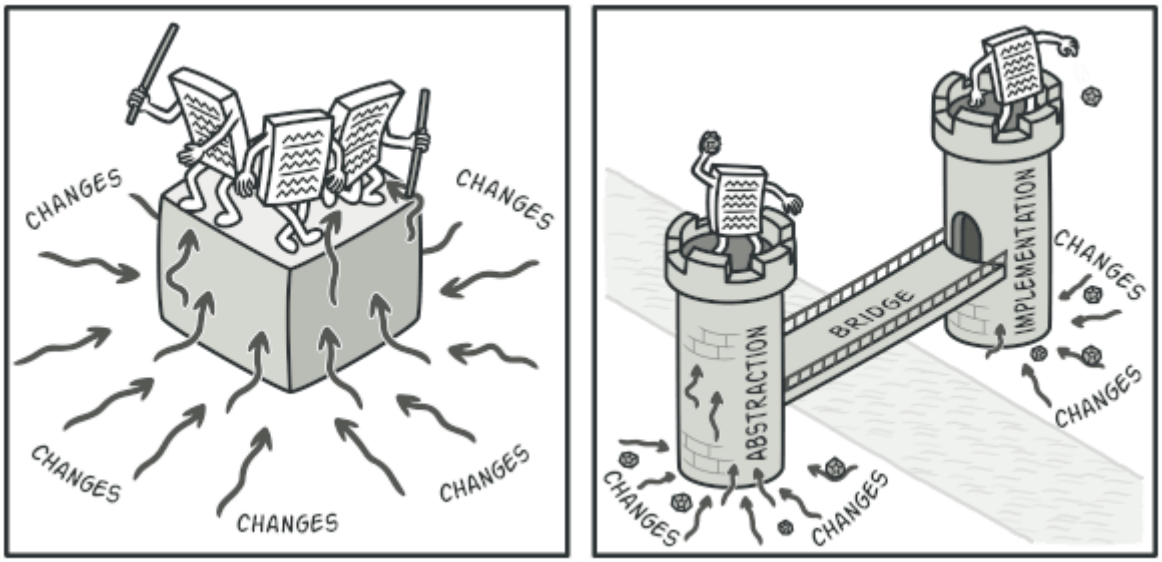
Beachten Sie, dass wir hier nicht über *Schnittstellen* oder *abstrakte Klassen* aus Ihrer Programmiersprache sprechen. Das sind nicht dieselben Dinge.

Wenn es um echte Anwendungen geht, kann die Abstraktion durch eine grafische Benutzeroberfläche (GUI) dargestellt werden, und die Implementierung könnte der zugrunde liegende Betriebssystemcode (API) sein, den die GUI-Schicht als Reaktion auf Benutzerinteraktionen aufruft.

Grundsätzlich kann man eine solche App in zwei voneinander unabhängige Richtungen erweitern:

- Bieten Sie mehrere unterschiedliche GUIs an (beispielsweise zugeschnitten auf Stammkunden oder Administratoren).
- Unterstützt mehrere verschiedene APIs (z. B. um die App unter Windows, Linux und macOS starten zu können).

Im schlimmsten Fall könnte diese App wie eine riesige Spaghettischüssel aussehen, in der Hunderte von Bedingungen im gesamten Code verschiedene GUI-Typen mit verschiedenen APIs verbinden.

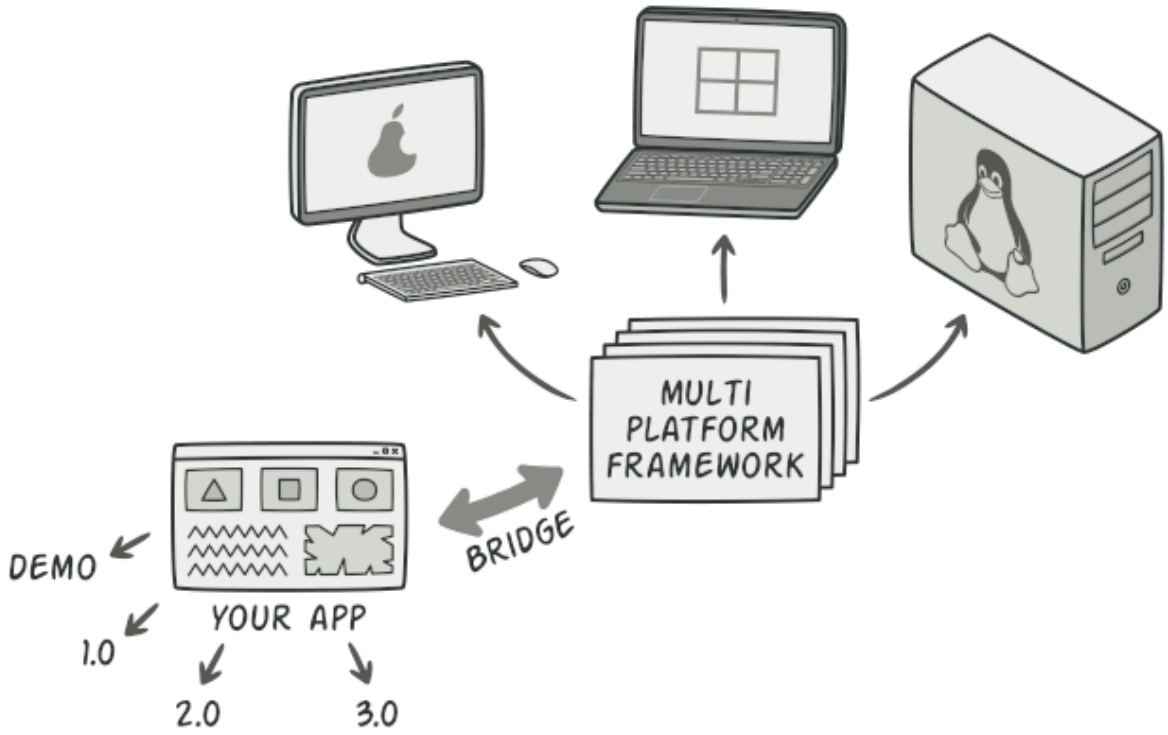


Selbst eine einfache Änderung an einer monolithischen Codebasis ist ziemlich schwierig, da man das *Ganze* sehr gut verstehen muss. Änderungen an kleineren, klar definierten Modulen sind deutlich einfacher.

Sie können Ordnung in dieses Chaos bringen, indem Sie den Code für bestimmte Schnittstellen-Plattform-Kombinationen in separate Klassen extrahieren. Sie werden jedoch bald feststellen, dass es *viele* dieser Klassen gibt. Die Klassenhierarchie wächst exponentiell, da das Hinzufügen einer neuen GUI oder die Unterstützung einer anderen API die Erstellung immer weiterer Klassen erfordern würde.

Versuchen wir, dieses Problem mit dem Bridge-Muster zu lösen. Es schlägt vor, die Klassen in zwei Hierarchien aufzuteilen:

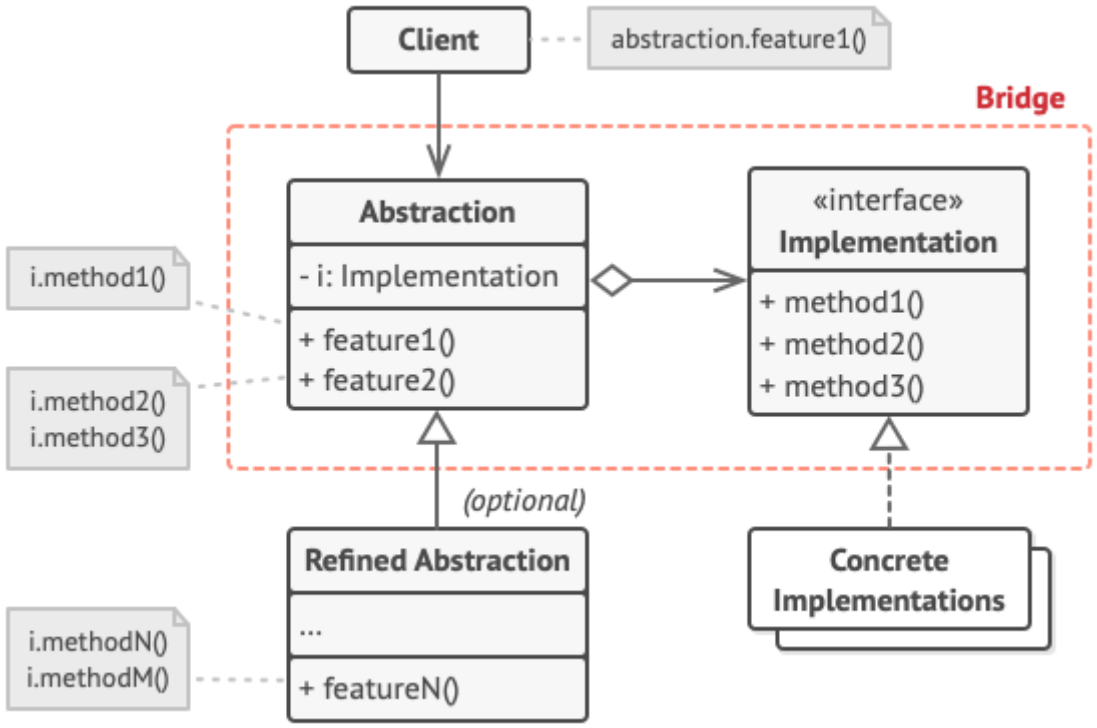
- Abstraktion: die GUI-Ebene der App.
- Implementierung: die APIs der Betriebssysteme.



Eine der Möglichkeiten, eine plattformübergreifende Anwendung zu strukturieren. Das Abstraktionsobjekt steuert das Erscheinungsbild der App und delegiert die eigentliche Arbeit an das verknüpfte Implementierungsobjekt. Verschiedene Implementierungen sind austauschbar, solange sie einer gemeinsamen Schnittstelle folgen, sodass dieselbe GUI unter Windows und Linux funktioniert.

Dadurch können Sie die GUI-Klassen ändern, ohne die API-bezogenen Klassen zu berühren. Darüber hinaus erfordert die Unterstützung für ein anderes Betriebssystem lediglich die Erstellung einer Unterklasse in der Implementierungshierarchie.

Struktur



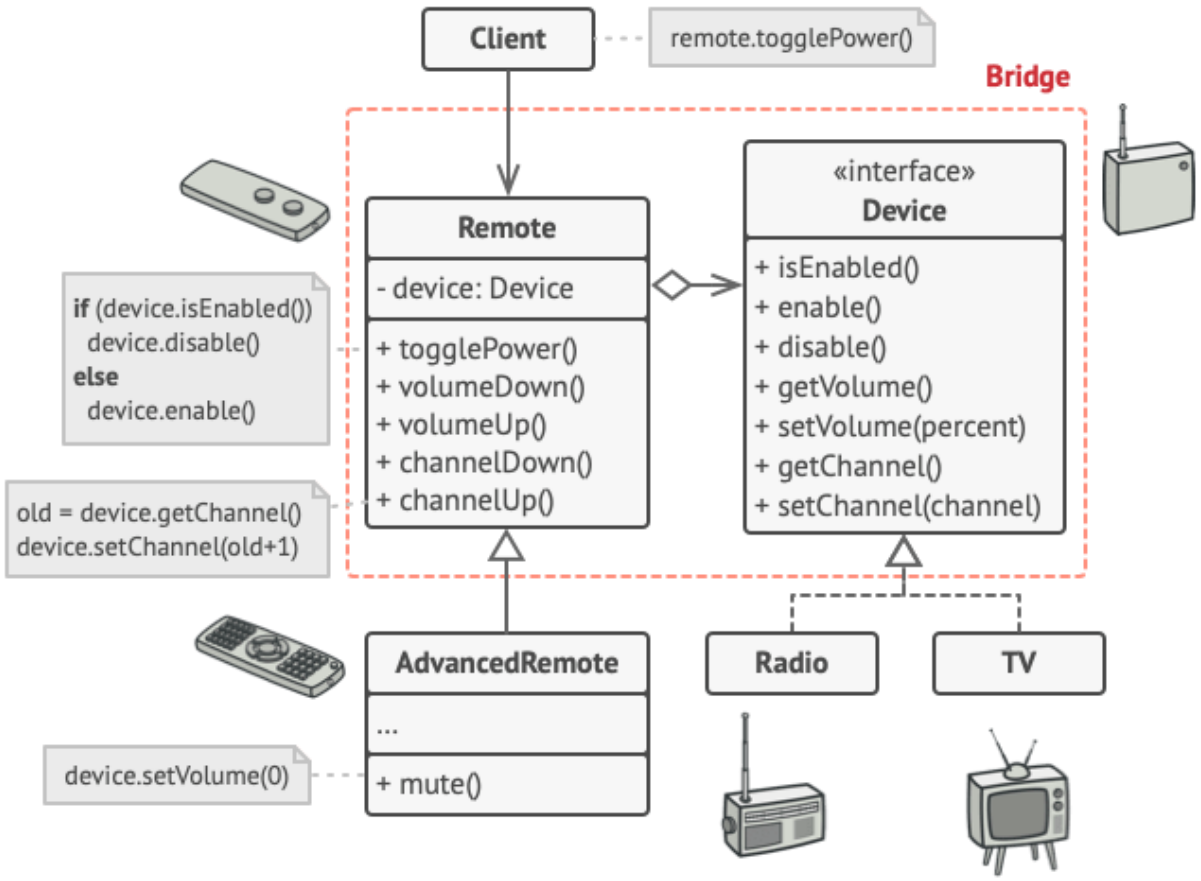
1. Die **Abstraktion** stellt eine übergeordnete Steuerungslogik bereit. Die eigentliche Arbeit auf niedrigerer Ebene wird vom Implementierungsobjekt übernommen.
2. Die **Implementierung** deklariert die Schnittstelle, die allen konkreten Implementierungen gemeinsam ist. Eine Abstraktion kann nur über hier deklarierte Methoden mit einem Implementierungsobjekt kommunizieren.

   Die Abstraktion kann dieselben Methoden auflisten wie die Implementierung, aber normalerweise deklariert die Abstraktion einige komplexe Verhaltensweisen, die auf einer Vielzahl primitiver Operationen beruhen, die von der Implementierung deklariert werden.

3. **Konkrete Implementierungen** enthalten plattformspezifischen Code.
4. **Verfeinerte Abstraktionen** bieten Varianten der Steuerungslogik. Wie ihre Vorgänger funktionieren sie mit verschiedenen Implementierungen über die allgemeine Implementierungsschnittstelle.
5. Normalerweise ist der **Client** nur daran interessiert, mit der Abstraktion zu arbeiten. Es ist jedoch seine Aufgabe, das Abstraktionsobjekt mit einem der Implementierungsobjekte zu verknüpfen.

Pseudocode

Dieses Beispiel veranschaulicht, wie das **Bridge** -Muster dabei helfen kann, den monolithischen Code einer App, die Geräte und deren Fernbedienungen verwaltet, aufzuteilen. Die `Device`Klassen fungieren als Implementierung, die `Remote`s als Abstraktion.



Die ursprüngliche Klassenhierarchie ist in zwei Teile unterteilt: Geräte und Fernbedienungen.

Die Basisklasse der Fernbedienung deklariert ein Referenzfeld, das sie mit einem Geräteobjekt verknüpft. Alle Fernbedienungen kommunizieren über die allgemeine Geräteschnittstelle mit den Geräten, sodass dieselbe Fernbedienung mehrere Gerätetypen unterstützt.

Sie können die Fernbedienungsklassen unabhängig von den Geräteklassen entwickeln. Dazu müssen Sie lediglich eine neue Remote-Unterklasse erstellen. Beispielsweise verfügt eine einfache Fernbedienung möglicherweise nur über zwei Tasten, lässt sich aber mit zusätzlichen Funktionen wie einem zusätzlichen Akku oder einem Touchscreen erweitern.

Der Clientcode verknüpft den gewünschten Fernbedienungstyp über den Konstruktor der Fernbedienung mit einem bestimmten Geräteobjekt.

```
// The "abstraction" defines the interface for the "control"
// part of the two class hierarchies. It maintains a reference
// to an object of the "implementation" hierarchy and delegates
// all of the real work to this object.
class RemoteControl is
    protected field device: Device
    constructor RemoteControl(device: Device) is
        this.device = device
    method togglePower() is
        if (device.isEnabled()) then
            device.disable()
        else
            device.enable()
    method volumeDown() is
        device.setVolume(device.getVolume() - 10)
    method volumeUp() is
        device.setVolume(device.getVolume() + 10)
    method channelDown() is
        device.setChannel(device.getChannel() - 1)
    method channelUp() is
        device.setChannel(device.getChannel() + 1)


// You can extend classes from the abstraction hierarchy
// independently from device classes.
class AdvancedRemoteControl extends RemoteControl is
    method mute() is
        device.setVolume(0)


// The "implementation" interface declares methods common to all
// concrete implementation classes. It doesn't have to match the
// abstraction's interface. In fact, the two interfaces can be
// entirely different. Typically the implementation interface
// provides only primitive operations, while the abstraction
// defines higher-level operations based on those primitives.
interface Device is
    method isEnabled()
    method enable()
    method disable()
    method getVolume()
    method setVolume(percent)
    method getChannel()
    method setChannel(channel)


// All devices follow the same interface.
class Tv implements Device is
    // ...

class Radio implements Device is

 // ...
// Somewhere in client code.
```

```
tv = new Tv()
remote = new RemoteControl(tv)
remote.togglePower()

radio = new Radio()
remote = new AdvancedRemoteControl(radio)
```

Anwendbarkeit

Verwenden Sie das Bridge-Muster, wenn Sie eine monolithische Klasse aufteilen und organisieren möchten, die über mehrere Varianten einer bestimmten Funktionalität verfügt (z. B. wenn die Klasse mit verschiedenen Datenbankservern arbeiten kann).
Je größer eine Klasse wird, desto schwieriger ist es, ihre Funktionsweise zu verstehen, und desto länger dauert eine Änderung. Änderungen an einer Funktionsvariante können Änderungen an der gesamten Klasse erfordern, was häufig zu Fehlern oder der Nichtbeachtung kritischer Nebeneffekte führt.

Mit dem Bridge-Muster können Sie die monolithische Klasse in mehrere Klassenhierarchien aufteilen. Anschließend können Sie die Klassen in jeder Hierarchie unabhängig von den Klassen in den anderen Hierarchien ändern. Dieser Ansatz vereinfacht die Codewartung und minimiert das Risiko, vorhandenen Code zu beschädigen.

Verwenden Sie das Muster, wenn Sie eine Klasse in mehreren orthogonalen (unabhängigen) Dimensionen erweitern müssen.
Die Bridge schlägt vor, für jede Dimension eine eigene Klassenhierarchie zu extrahieren. Die ursprüngliche Klasse delegiert die zugehörige Arbeit an die Objekte dieser Hierarchien, anstatt alles selbst zu erledigen.

Verwenden Sie die Bridge, wenn Sie Implementierungen zur Laufzeit wechseln müssen.
Obwohl optional, ermöglicht das Bridge-Muster das Ersetzen des Implementierungsobjekts innerhalb der Abstraktion. Dies ist so einfach wie das Zuweisen eines neuen Werts zu einem Feld.

Dieser letzte Punkt ist übrigens der Hauptgrund, warum so viele Leute das Bridge- mit dem **Strategy-** Muster verwechseln. Bedenken Sie, dass ein Muster mehr ist als nur eine bestimmte Art, Ihre Klassen zu strukturieren. Es kann auch die Absicht und das zu lösende Problem kommunizieren.

So implementieren Sie

1. Identifizieren Sie die orthogonalen Dimensionen Ihrer Klassen. Diese unabhängigen Konzepte könnten sein: Abstraktion/Plattform, Domäne/Infrastruktur, Front-End/Back-End oder Schnittstelle/Implementierung.
2. Sehen Sie, welche Operationen der Client benötigt, und definieren Sie sie in der Basisabstraktionsklasse.
3. Bestimmen Sie die auf allen Plattformen verfügbaren Operationen. Deklarieren Sie diejenigen, die die Abstraktion benötigt, in der allgemeinen Implementierungsschnittstelle.

4. Erstellen Sie für alle Plattformen in Ihrer Domäne konkrete Implementierungsklassen, stellen Sie jedoch sicher, dass sie alle der Implementierungsschnittstelle folgen.
5. Fügen Sie innerhalb der Abstraktionsklasse ein Referenzfeld für den Implementierungstyp hinzu. Die Abstraktion delegiert den Großteil der Arbeit an das Implementierungsobjekt, auf das in diesem Feld verwiesen wird.
6. Wenn Sie über mehrere Varianten der Logik auf hoher Ebene verfügen, erstellen Sie für jede Variante verfeinerte Abstraktionen, indem Sie die Basisabstraktionsklasse erweitern.
7. Der Clientcode sollte ein Implementierungsobjekt an den Konstruktor der Abstraktion übergeben, um beide miteinander zu verknüpfen. Danach kann der Client die Implementierung vergessen und nur noch mit dem Abstraktionsobjekt arbeiten.

Für und Wider

- Sie können plattformunabhängige Klassen und Apps erstellen.
- Der Clientcode arbeitet mit Abstraktionen auf hoher Ebene. Er ist den Plattformdetails nicht ausgesetzt.
- *Offenes/Geschlossenes Prinzip* . Sie können neue Abstraktionen und Implementierungen unabhängig voneinander einführen.
- *Single Responsibility Principle* : Sie können sich bei der Abstraktion auf die Logik auf höherer Ebene und bei der Implementierung auf Plattformdetails konzentrieren.

- Sie können den Code komplizierter gestalten, indem Sie das Muster auf eine stark kohäsive Klasse anwenden.

Beziehungen zu anderen Mustern

- **Bridge wird in der Regel im Voraus entwickelt, sodass Sie Teile einer Anwendung unabhängig voneinander entwickeln können. Adapter** hingegen wird häufig mit einer vorhandenen App verwendet, um die reibungslose Zusammenarbeit ansonsten inkompatibler Klassen zu gewährleisten.
- **Bridge** , **State** , **Strategy** (und bis zu einem gewissen Grad **auch Adapter**) weisen sehr ähnliche Strukturen auf. Tatsächlich basieren alle diese Muster auf Komposition, also der Delegierung von Arbeit an andere Objekte. Sie lösen jedoch alle unterschiedliche Probleme. Ein Muster ist nicht nur ein Rezept für die Strukturierung Ihres Codes. Es kann auch anderen Entwicklern das Problem vermitteln, das das Muster löst.
- **Sie können Abstract Factory** zusammen mit **Bridge** verwenden . Diese Kombination ist nützlich, wenn einige von *Bridge* definierte Abstraktionen nur mit bestimmten Implementierungen funktionieren. In diesem Fall kann *Abstract Factory* diese Beziehungen kapseln und die Komplexität vor dem Clientcode verbergen.
- **Sie können Builder** mit **Bridge** kombinieren : Die Director-Klasse spielt die Rolle der Abstraktion, während verschiedene Builder als Implementierungen fungieren.